Verilog (week 3): some assembly required

Recap slides follow

Back to Basics: Module Definition

- Module definition syntax

```
module priority_decoder_4to2 (
    input  wire [3:0] in,         // 4 input request lines
    output reg  [1:0] code,       // encoded index of highest-priority input
    output reg        valid       // 1 if any input is asserted
);…

endmodule
```

  - `input`/`output` are keywords to define direction of port (`inout` port can be used for both)
  - `// …` and `/* */` are for comments
  - `wire`/`reg` define data types and `[N:0]` defines size; we will get to them later
  - Each module definition must define ports (inputs & outputs) in parentheses
  - Module definition ends with `endmodule`
  - **Note:** Curly braces are replaced with explicit `begin`, `end...` -style statements in Verilog

Back to Basics: Instantiation

- define variables for **input/output**

  for eg: `req_lines, priority_code, req_valid`

- instantiate it with a **unique instance name**

  for eg: `pdec_inst` is the unique identifier for this instance of `priority_decoder_4to2`

- wire in the inputs/outputs using **port names**

  for eg: `.in(req_lines)` wires in `req_lines` to the `in` port

```
reg  [3:0] req_lines;     // input - reg
wire [1:0] priority_code; // output - wire
wire       req_valid;     // output - wire

priority_decoder_4to2 pdec_inst (
    .in    (req_lines),
    .code  (priority_code),
    .valid (req_valid)
);
```

Back to Basics: Can you C it?

Most C-style **operators** are supported:

- Bitwise: `& | ^ ~` — per-bit operations
- Logical: `&& || !` — collapse to true/false, used in conditions
- Comparison: `== != < >`
- Concatenation: `{a, b}` — combines signals into a wider signal vector

Control constructs (allowed inside `always` / `initial` only)

- `if / else` — same as software
- `case` — switch-statement for hardware

Back to basics: One is^{n}t gud genug

Using vector signals in verilog is easy:

- **Declaring:** `wire/reg [MSB:LSB] var_name`

  for eg: `wire [3:0] data` declares a wire that can carry 4 bits

- **Literals:** `<width>'<base><value>`

  for eg: `4'b1010` (binary), `8'hA5` (hex), `4'd9` (decimal)

- **Assignment**:

  `data = 4'b0110` (full assignment) `data[1:0] = 2'b10` (multi-bit assignment)

  `data[3:0] = {2'b01, 2'b10}` (using concat operator)

  > this will assign the bits 3, 2, 1, 0 (left to right) to the values 0, 1, 1, 0.

- **Reading:** Same syntax as assignment

  for eg: `data[high:low]` will read bits `low,low+1,...high-1,high`

- Note: You can use the `[LSB:MSB]` convention as well in Verilog – stay consistent with one convention.

Back to Basics: More data types

Verilog provides helper data types to make life easier – these do not map to real hardware (`wire/reg` do).

- **`integer`** – plain whole number variable, mostly used to count things (like loop iterations)

  for eg: `integer i; for (i = 0; i < 16; i = i + 1)`.

- **`parameter`** — a named constant you can **reconfigure** from outside the module

  for eg: `parameter WIDTH = 8` – can write variable-width modules using this

- `real` — a decimal/floating-point number; **only used in simulation/testing**

  for eg: `real voltage = 3.3`.

- `time` — stores a **simulation/testing timestamp**, usually paired with `$time`

  for eg: `time start_time; start_time = $time;`.

- `localparam` — a named constant that is **fixed inside a module** – used to store helper values

  for eg: `localparam OP_ADD = 4'b0000;`.

Theory Terms

- Recall the following from CS230:
  - **Clock**: A periodic signal (typically square wave) that triggers updates in sequential circuits.
  - **Level-triggered**: A circuit that responds continuously **while a control signal is high** (or low).
  - **Edge-triggered**: A circuit that responds only at **an instant** — when the clock transitions from **0→1 (posedge)** ..... or ..... **1→0 (negedge)**

  - **D Flip-Flop (edge-triggered)**: A circuit that samples its input on a clock edge and holds that value until the next edge
  - **T Flip-Flop (edge-triggered)**: A circuit that toggles its output (flips 0↔1) when both the clock edge arrives and the toggle input is 1

Back to basics: Data Types

Verilog supports two data types for these purposes:

- `wire` – carries a value **continuously** from whatever drives it; has no memory of its own
  - `assign` is used for **continuous assignment** to a wire. It cannot be used for register data.
  - Eg: `assign q_bar = ~q_reg;`

- `reg` — a variable that **holds its value** until explicitly assigned inside a procedural block
  - **Note:** The definition is **logical**. It is not guaranteed to synthesize to a **physical register**
  - Can only be assigned inside of `always @(...)` blocks
  - Usage: `always @(trigger-list)`
    - Eg: `always @(posedge clk)` executes on **rising edge of `clk` signal -> sequential**
    - Eg: `always @(*)` executes whenever **any input changes -> combinational**

Coming back to you (always)

Combinational

- requires **no physical registers**
- **ALL updates are executed at once**
- (typically) blocking assignment (`=`) is used
- written inside `always @(*)` blocks
- the blocking statements run in **program order**
- Eg: some random combinational logic here

```
always @(*) begin
    c = a & b;
    d = a | b;
    out1 = d;
    out2 = c ^ b;
end
```

Sequential

- (usually) requires **physical registers**
- updates transferred **sequentially ONE-by-ONE**
- (typically) non-blocking assignment (`<=`) is used
- written inside `always @(posedge clk)` blocks (or a variant)
- Eg: naive 2-bit (modulo 4) counter

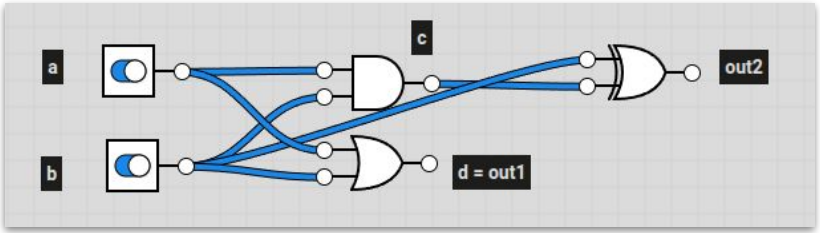
```
always @(posedge clk) begin
    if (reset)
        count <= 2'b00;
    else
        count <= count + 2'b01;
end
```

Coming back to you (always)

Combinational

```
always @(*) begin
    c = a & b;
    d = a | b;
    out1 = d;
    out2 = c ^ b;
end
```

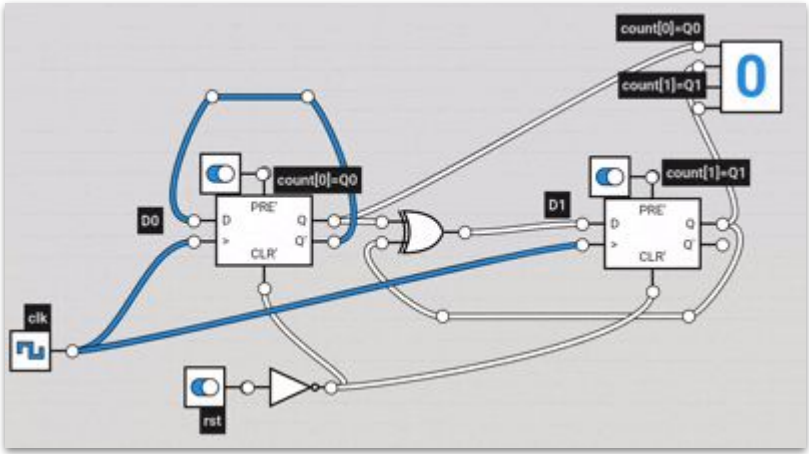
- Circuit will respect the program order for computation



Sequential

```
always @(posedge clk) begin
    if (reset)
        count <= 2'b00;
    else
        count <= count + 2'b01;
end
```

- Circuit will respect the clock-based update



How NOT to TFF it!

1. `tff_wire`

   Need **reg** not **wire**; Need **edge-triggered** not **level-triggered**

```
module tff_wire (
    input clk,
    input t,
    output wire q
);
    assign q = clk ? (t ^ q) : q;
endmodule
```

2. `tff_reg`

   Blocking assignment gives wrong assignment, need **<=**

```
module tff_reg (
    input clk,
    input t,
    output reg q
);
    always @(posedge clk) begin
        q = t ^ q;
    end
endmodule
```

Why you should not block (me)

- `<=` stands for non-blocking assignment – this allows us to defer updates to the end and execute them together

- How it works at a high level:
  - Read all old values (occurring on right hand side first
  - Then update all left hand side values **together**

- Why is it called “non-blocking”?

  All writes are **scheduled** to happen later, i.e., it **doesn’t block** any other statement trying to read the **old value**.

  A blocking assignment (`=`) will write the new value immediately and block anyone else trying to read the old value.

- **Rule of thumb:** Always use `<=` inside `always @(posedge clk)` or some variant.

- **Note: `<=` and `=` (other than `assign <wire> =` statements) can only be used in `always` or `initial` blocks.**

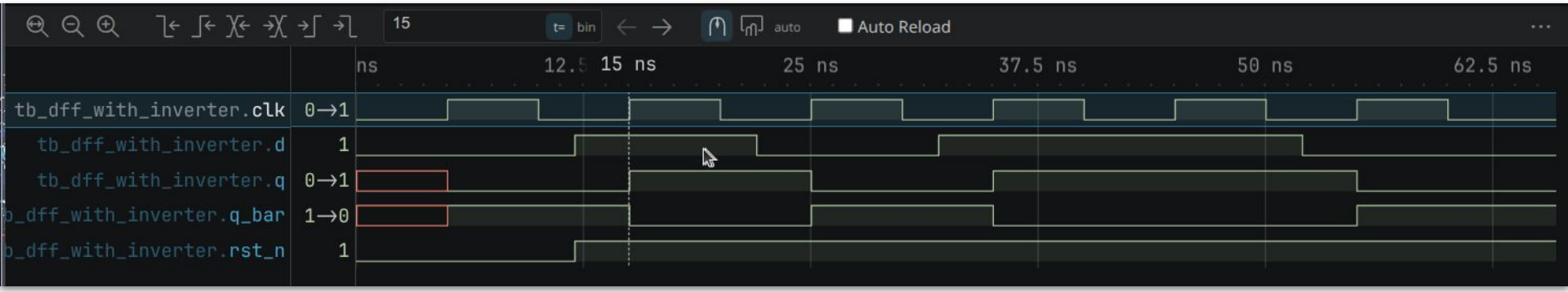
Example: DFF

```
module dff_with_inverter (
    input  wire clk,
    input  wire rst_n,
    input  wire d,
    output wire q,
    output wire q_bar
);

    reg q_reg;

    // ---- Sequential logic ----
    // Output depends on current input AND past state (memory)
    // Only updates on the clock's rising edge
    always @(posedge clk) begin
        if (!rst_n)
            q_reg <= 1'b0;
        else
            q_reg <= d;
    end

    // ---- Combinational logic ----
    // Output depends only on current input, no memory, no clock
    assign q     = q_reg;
    assign q_bar = ~q_reg;

endmodule
```

Example: DFF testbench

```
`timescale 1ns/1ns

module tb_dff_with_inverter;

    reg  clk, rst_n, d;
    wire q, q_bar;

    dff_with_inverter dut (
        .clk(clk), .rst_n(rst_n),
        .d(d), .q(q), .q_bar(q_bar)
    );

    // Clock generation: 10ns period
    always #5 clk = ~clk;

    initial begin
        $dumpfile("dff_wave.vcd");
        $dumpvars(0, tb_dff_with_inverter);

        clk = 0; rst_n = 0; d = 0;

        #12 rst_n = 1;   // release reset

        d = 1; #10;
        d = 0; #10;
        d = 1; #10;
        d = 1; #10;
        d = 0; #10;

        $display("Test complete.");
        $finish;
    end

    initial begin
        $monitor("t=%0t rst_n=%b d=%b q=%b q_bar=%b",
                 $time, rst_n, d, q, q_bar);
    end

endmodule
```

Example: DFF Waveform



- An example testbench waveform for the D-flip flop implementation
- Note clock time period is 10ns
- `d` update is propagated to `q` only on **rising edge** when `rst_n` is set

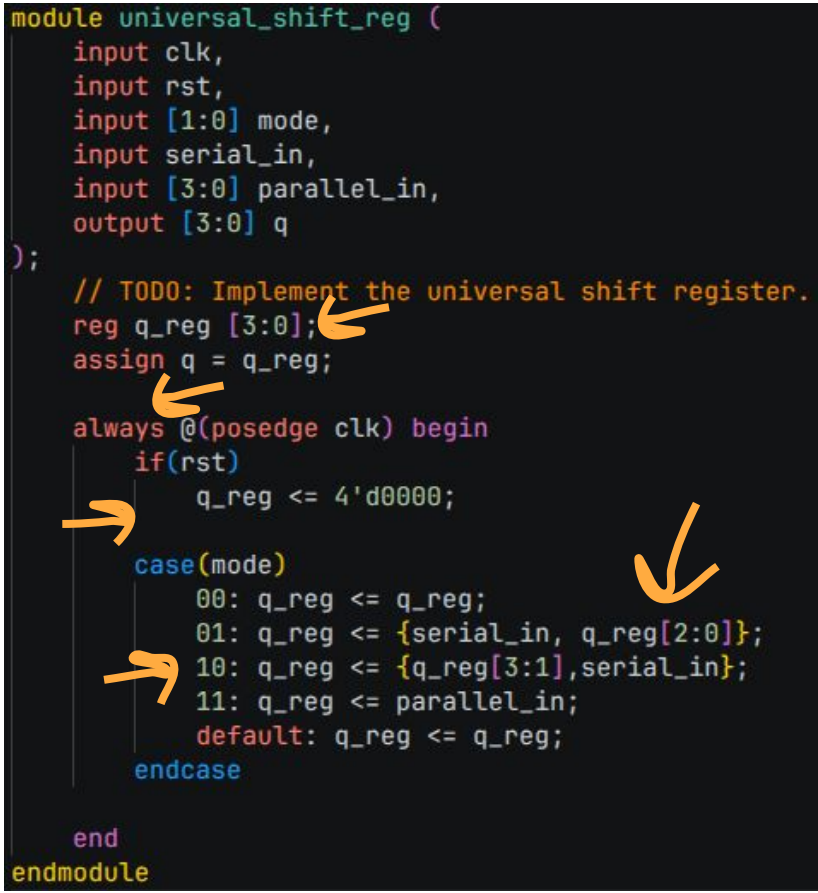
Buffer Slide

Discussing doubts/solutions attempts for last week's inlab / labexam / outlab

These should be doubts

Common problems you guys ran into in the last labexam:

1. Use `reg [3:0] q_reg` – **Single 4-bit register – Better choice**
   NOT `reg q_reg [3:0]` – an array of 1-bit regs
2. A `if` and `case` always occur inside an `always` block
3. Binary literals **need to be preceded with `2'b`** here
4. Do not miss the `else` **clause** here
5. Be careful with **bit indexing** for left and right shift
6. Don't run `initial q = 4'b0` in the module, let `rst` handle it



```
module universal_shift_reg (
    input clk,
    input rst,
    input [1:0] mode,
    input serial_in,
    input [3:0] parallel_in,
    output [3:0] q
);
    // TODO: Implement the universal shift register.
    reg q_reg [3:0];
    assign q = q_reg;

    always @(posedge clk) begin
        if(rst)
            q_reg <= 4'd0000;

        case(mode)
            00: q_reg <= q_reg;
            01: q_reg <= {serial_in, q_reg[2:0]};
            10: q_reg <= {q_reg[3:1],serial_in};
            11: q_reg <= parallel_in;
            default: q_reg <= q_reg;
        endcase

    end
endmodule
```

PS: Resetting Your Counts to Zero

- **Reset**: A signal used to reset the register values to zero

- **Synchronous Reset**: Only takes effect when clock signal permits (for eg on posedge clk)
  - When we used `always @(posedge clk)` and `if(rst)`, we were using this pattern

- **Asynchronous Reset**: Takes effect whenever updated – independent of clock signal
  - More common in practice. Used as `always @(posedge clk or posedge rst)`

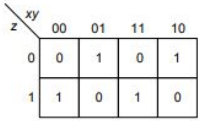
- The above is called an **"active-high" reset** which resets when the `rst` signal goes **high**
- In practice, **"active-low" reset** is more common, i.e., it resets when the `rst_n` signal goes **low**
- Use it as `always @(posedge clk or negedge rst_n)` -- we will use this more frequently

PS: Know your geography

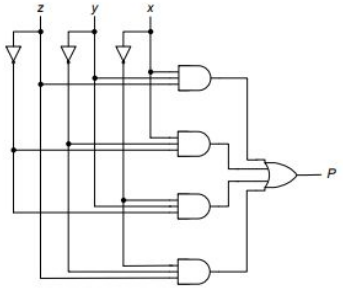
- Recall Karnaugh maps
- Why do we use them?
- What are don't-cares?
- Can you simplify boolean functions?

Combinational Circuits: Parity-bit Generator

Parity-bit generator: produces output value 1 if and only if an odd number of its inputs have value 1



(*a*) Map.



(*b*) Implementation.

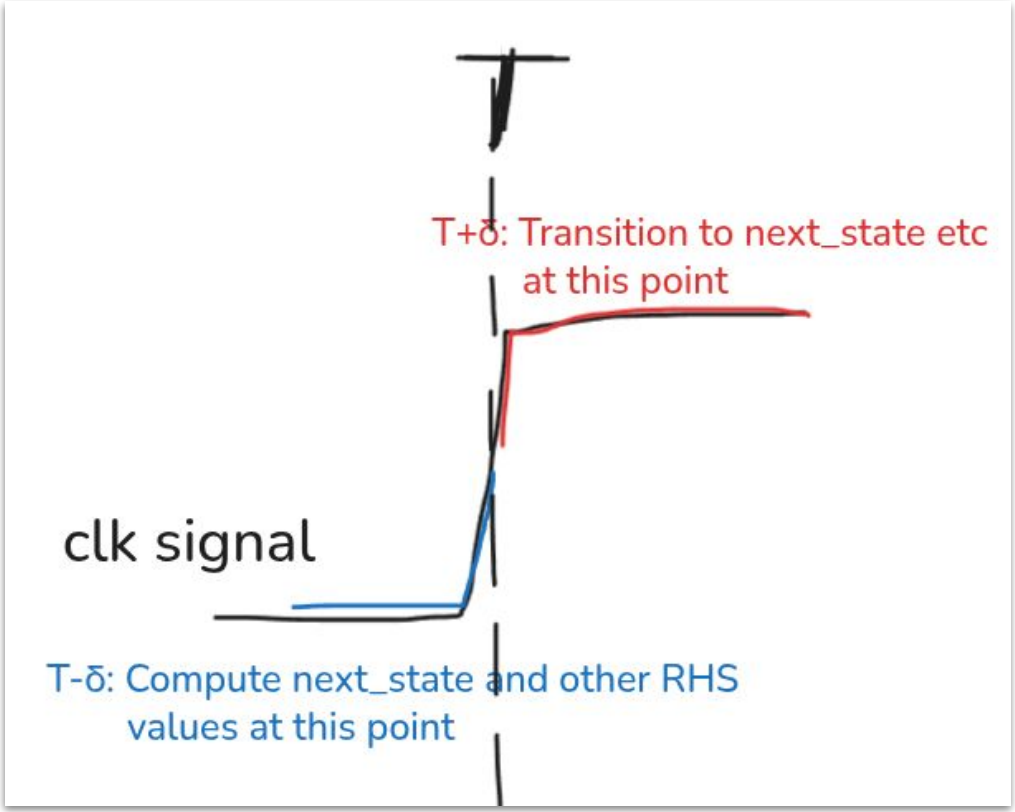
$$P = x'y'z + x'yz' + xy'z' + xyz = x \oplus y \oplus z$$

- Practice?

1. Simplify the following Boolean function using a Karnaugh Map and express the final result in the minimal Sum of Products (SOP) form:

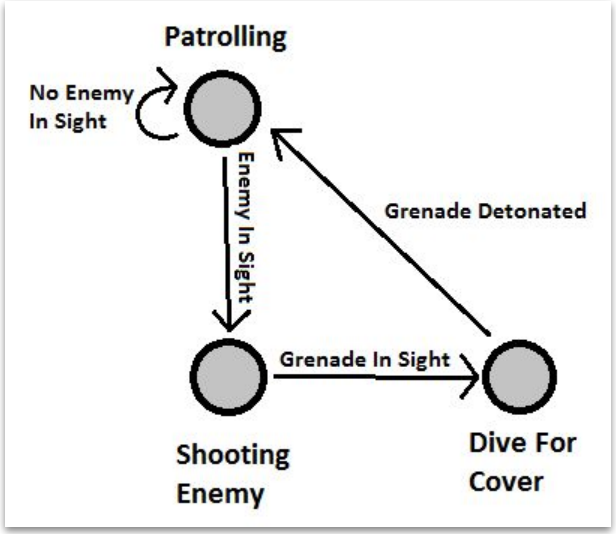
$$F(A, B, C, D) = \sum m(0, 2, 5, 7, 8, 10, 13, 15)$$

PS: Clocked Sequential Circuits FTW

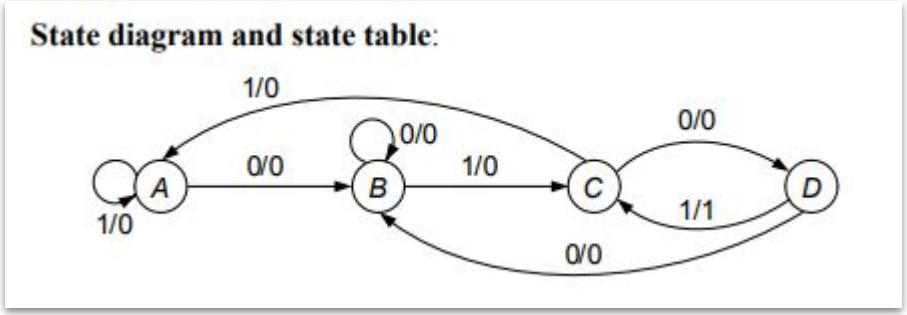


Why more Verilog?

- We will cover hardware design from a high level today!
- Also a common idiom in hardware design: finite state machines.
- **Finite state machines** define well-defined states (memory) and transitions (combinational logic) between them to consume some input and produce some output.
- Example: Bot AI in games be like



Eg: Simple 0101 detector



Plan for this lab

- Designing and writing a simple FSM-based circuit together
- First discuss, later you attempt as task1

- **Goals:**
  - Convert specifications to **high-level component design**
  - Write an **FSM** in Verilog: states, transitions, outputs, counters
  - **Combine smaller modules** to solve the entire problem

Traffic Control Simulator

P1. Bhavesh wants to go to his favourite restaurant right across JVLR road. But the traffic is horrible. Bhavesh is annoyed and blames the traffic light controller. He wishes to design a better traffic control system!

Goal: Build a chip that cycles through green->yellow->red phases with some fixed duration for each, and takes a pedestrian button as input – when pressed by a pedestrian, reduce the duration of green and increase that of red.

- **Q1**: What external inputs do we need? tick tock...
- **Q2:** What outputs do we need to produce?
- **Q3:** Do we need to store state for the circuit?
- **Q4**: What are the transitions, and when do they happen (ignore the pedestrian button for now)?
- **Q5:** Now add the pedestrian button. What should happen, precisely?
- **Q6:** Can we separate control from datapath? What goes where, and why?

Durations are as follows:

- *GREEN: Normally, **25** / If pedestrian request is on, **15***
- *RED: Normally, **30** / If pedestrian request is on, **40***
- *YELLOW: Always **5***

Traffic Control Simulator

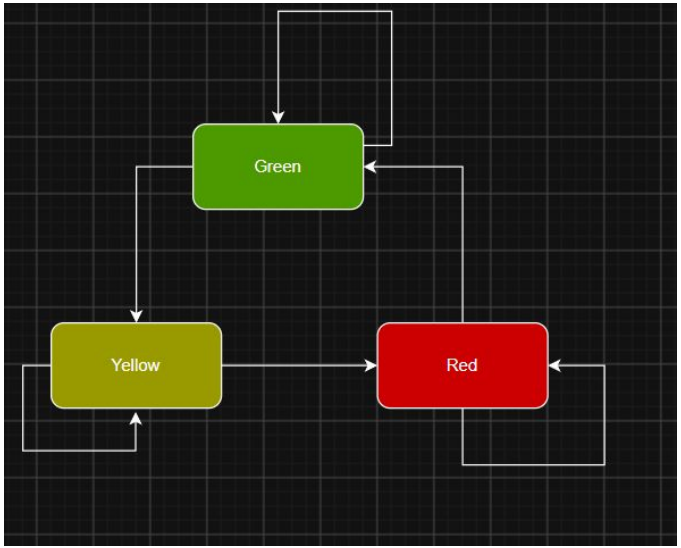
- **Q1**: What external inputs do we need? tick tock...
  - Sequential/state-based circuit -> We need **clk, rst_n**
  - Button for pedestrian -> **ped_button**
  - Is clk a good way to keep track of time? Why/why not?
    - We need **tick_1hz**

- **Q2:** What outputs do we need to produce?
  - One wire for each output: **red**, **yellow**, **green**

```
module bhavesh_traffic_light (
  input clk, rst_n,
  input ped_button,
  input tick_1hz,

  output red, green, yellow,
);
```

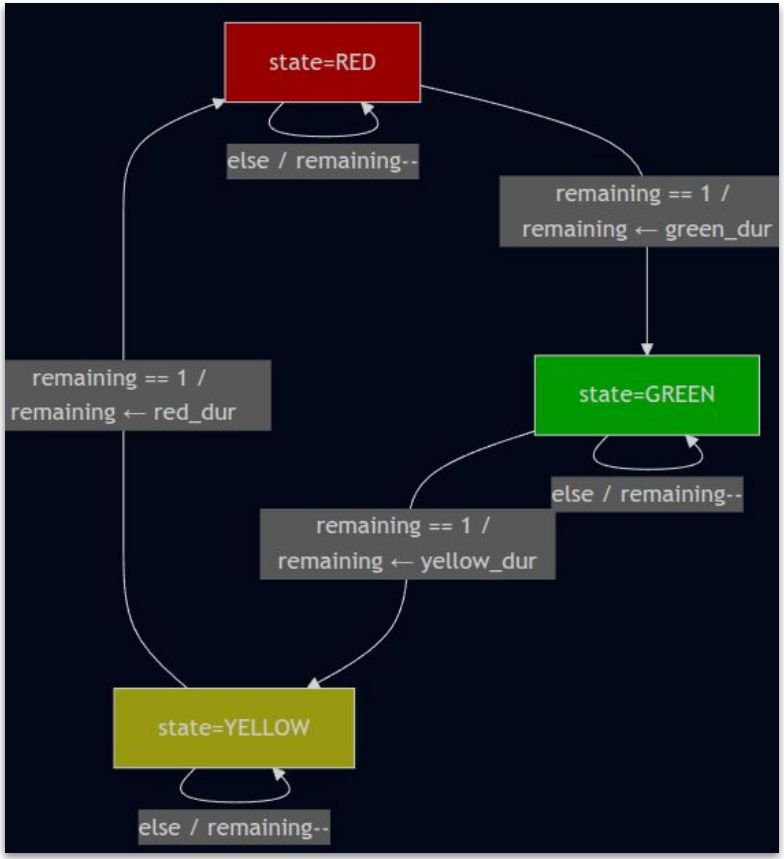
Traffic Control Simulator

- **Q3:** Do we need to store state for the circuit?
  - Yes. We need 3 states - red, yellow and green
  - Are we designing a **Moore** or **Mealy** machine?
    - Moore!
- **Q4:** What are the transitions, and when do they happen (ignore the pedestrian button for now)?
  - Green : Can stay at Green or go to Yellow
  - Yellow : Can stay at Yellow or go to Red
  - Red: Can stay at Red or go to Green



  - This is the **design**, how do we **implement** it?
  - Need a counter (**remaining**) that counts down for each state
  - We switch states when the counter is **at 1** (not 0) Why?

Abstraction!

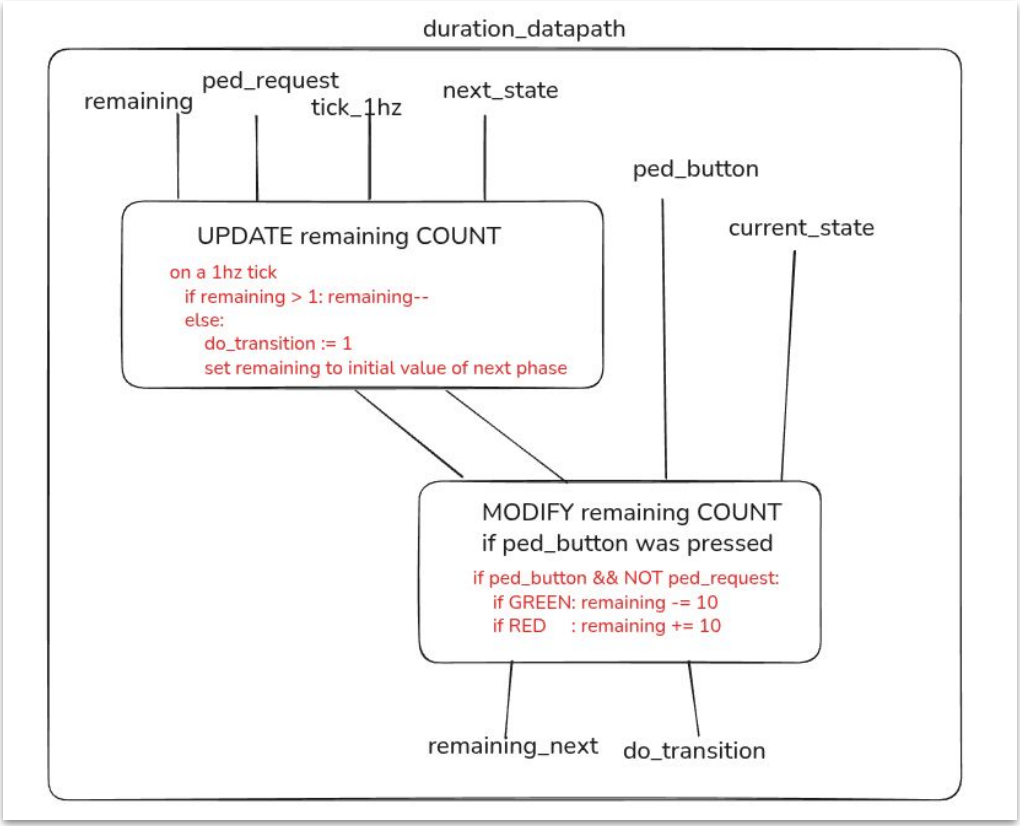


Traffic Control Simulator

Q5: Now add the pedestrian button. What should happen, precisely?

- Writing down intended behaviours on pressing button:
  a. Pressed during GREEN: GREEN durations shrinks (-10), RED extends (+10)
  b. Pressed during YELLOW or RED: RED extends (+10) because GREEN for this iteration is gone
- We need two behaviours:
  1. Instantly shrink/extend duration if the button press was during GREEN or RED phase
  2. "Remember" the press of a button until the end of this loop **(G->Y->R is a loop)**.
- For 1, when button is pressed, check if **state ∈ {GREEN, RED}** and do **-10/+10** to **remaining**
- For 2, latch the button until end of loop, and do a **+10** if transitioning **to RED** (from YELLOW)
  - Call the latch **ped_request**
- How to prevent long button press from triggering multiple -/+10s on GREEN?
  - Use **ped_button && !ped_request** for instantaneous triggering
- Did we miss something?
  - **Reset ped_request** after a transition out of RED

Traffic Control Simulator



Traffic Control Simulator

Q6: Can we separate control from datapath? What goes where, and why?

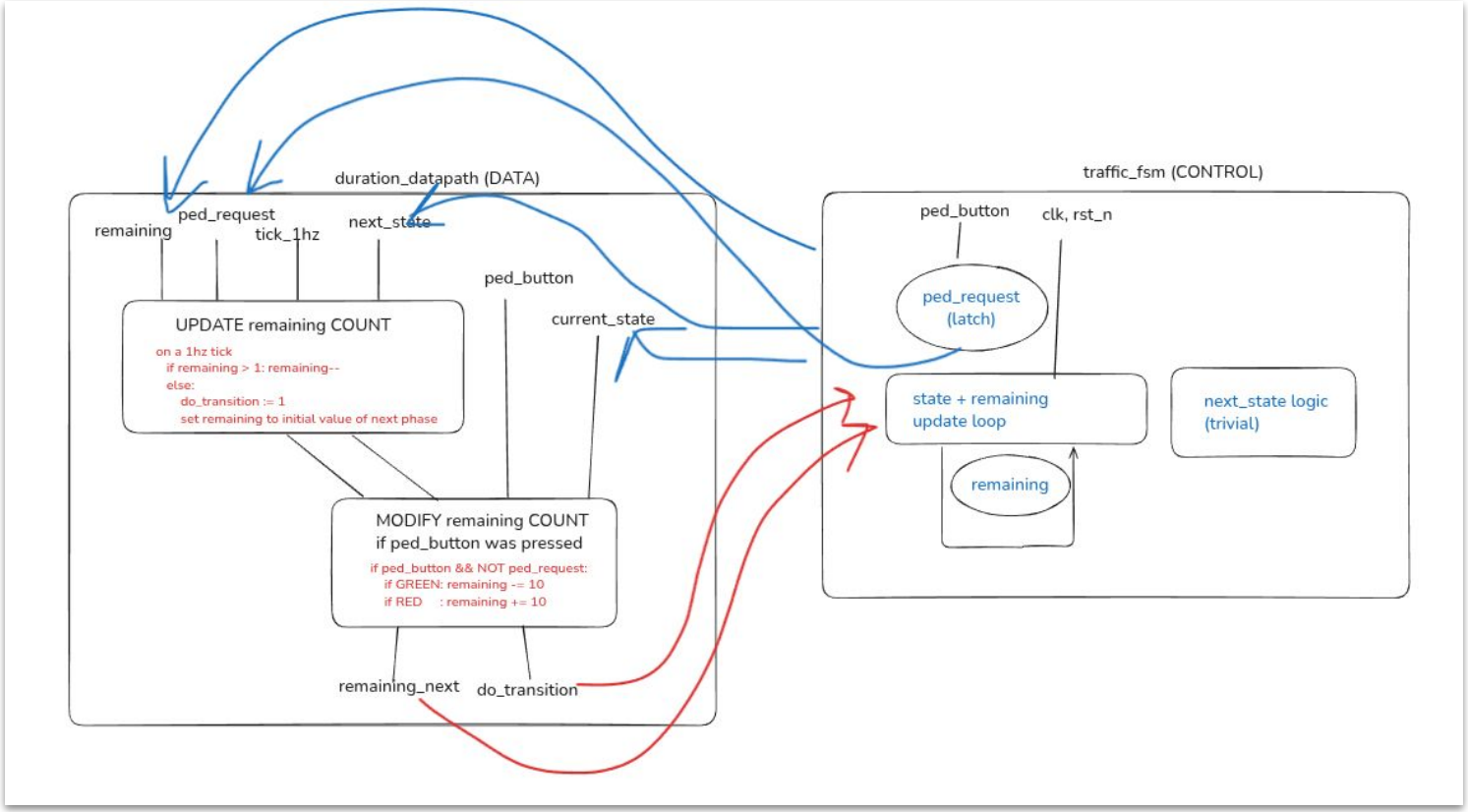
- What are the control signals here?
  - **tick_1hz**, **ped_button**, **ped_request**
- What is the “data” here?
  - **remaining** counter and **current_state**, **next_state**
- We separate the counter-tracking and updating logic entirely (datapath) into **duration_datapath**
  - Handles timer arithmetic, dealing with ped_button (fresh and old) – **combinational**
- We keep the control signals and state tracking inside **traffic_fsm**
  - Handles the overall state transitions based on reading/writing control signals – **sequential**

The two modules lived happily ever after?

Few extra questions and notes:

- We need a **do_transition** control signal to switch states... Who should produce this signal?
  - **duration_datapath!** Why?
- Do we need to store **next_state**?
  - Yes.
- Who should handle the latching of **ped_request** and computing **next_state**?
  - **traffic_fsm!**
- What should happen if someone presses ped_button on a “transition tick” (the final tick of a phase, at which we are going to switch states)? What issues can emerge?
  - Verdict: ignore these presses for now

Two modules happy ending



Plan for this lab

- Designing and writing a simple FSM-based circuit together

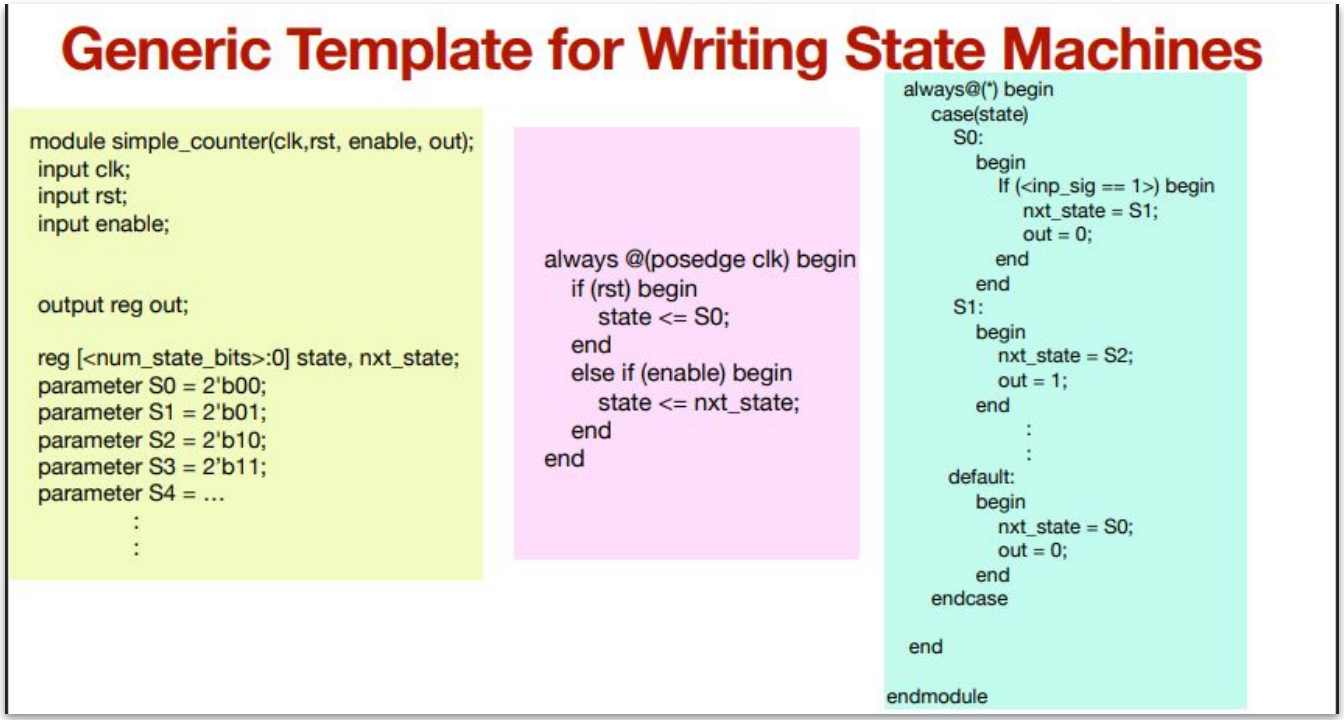
- **Goals:**
  - ~~Convert specifications to **high-level component design**~~
  - Write an **FSM** in Verilog: states, transitions, outputs, counters
  - **Combine smaller modules** to solve the entire problem

Recipe to a great FSM

1. Write one clocked, sequential `always` block to **model state updates**
2. Use `case` statement to **write the transitions/next_state for every state**
3. Use `<=` to apply changes to a **state register**

Recipe to a great FSM: Life is simple

- Recall this from CS230: don't read it in detail, get the high-level idea



Recipe to a great FSM

Do it on the board / VS Code!

Plan for this lab

- Designing and writing a simple FSM-based circuit together

- **Goals:**
  - ~~Convert specifications to **high-level component design**~~
  - ~~Write an **FSM** in Verilog: states, transitions, outputs, counters~~
  - **Combine smaller modules** to solve the entire problem

Wiring is hard

- Have 3 components:
  1. **duration_datapath:** Inputs current **remaining** counter, curr/next **state**, **ped btn/req** and outputs updated **remaining_next** counter and **do_transition** signal
  2. **output_decoder:** Inputs **state** and outputs high on the wire corresponding to that state
  3. **traffic_fsm: (actual logic)** Inputs **clk, rst_n, ped_button, remaining_next** counter, **do_transition** signal and outputs **current_state, next_state, remaining** counter and **ped_request**

- Initialize all 3 and wire them up

- High-level module takes in inputs **clk, rst_n, tick_1hz, ped_button** and outputs **red/green/yellow**

Wiring it all up

```
module bhavesh_traffic_light (
    input clk, rst_n,
    input ped_button,
    input tick_1hz,

    output [1:0] state_out,
    output [7:0] duration_out,
    output red, green, yellow
);

    wire [1:0] current_state, next_state;
    wire [7:0] remaining, remaining_next;
    wire       ped_request;
    wire       do_transition;
```

```
    traffic_fsm fsm (
        .clk(clk),
        .rst_n(rst_n),
        .ped_button(ped_button),
        .next_state(next_state),
        .remaining_next(remaining_next),
        .do_transition(do_transition),
        .current_state(current_state),
        .remaining(remaining),
        .ped_request(ped_request)
    );

    duration_datapath dp (
        .current_state(current_state),
        .next_state(next_state),
        .remaining(remaining),
        .tick_1hz(tick_1hz),
        .ped_button(ped_button),
        .ped_request(ped_request),
        .remaining_next(remaining_next),
        .do_transition(do_transition)
    );

    output_decoder decoder (
        .state(current_state),
        .red(red),
        .green(green),
        .yellow(yellow)
    );
```

It’s inlab (1) time

Alice in TestbenchLand

Read testbench output and view waveforms:

- Reduce durations while testing to get a much shorter trace.
- Remove `clk` from the `$monitor` format string to suppress the lines where only the clock toggles
- Read the testbench code itself. Understand what each test is doing, then look for that behaviour in the monitor output.

Plan for this lab

- Designing and writing a simple FSM-based circuit together

- **Goals:**
  - ~~Convert specifications to **high-level component design**~~
  - ~~Write an **FSM** in Verilog: states, transitions, outputs, counters~~
  - ~~**Combine smaller modules** to solve the entire problem~~

Plan for this lab

- Designing and writing a simple FSM-based circuit together

- **Goals:**
  - ~~Convert specifications to **high-level component design**~~
  - ~~Write an **FSM** in Verilog: states, transitions, outputs, counters~~
  - ~~**Combine smaller modules** to solve the entire problem~~
  - **genvar** and **parametric modules???**
    - **Optional (for outlab) and NOT in labexam**

Parametric is better

- A **`parameter`** is a constant you set at instantiation time. The module is written once, the width is filled in when you use it.
- Definition

```
module adder #(parameter N = 8) (
  input  [N-1:0] a, b,
  output [N-1:0] sum
);
  assign sum = a + b;
endmodule
```

- Usage

```
adder #(.N(8))  add8  (.a(a8),  .b(b8),  .sum(s8));
adder #(.N(16)) add16 (.a(a16), .b(b16), .sum(s16));
adder #(.N(32)) add32 (.a(a32), .b(b32), .sum(s32));
```

Looping over hardware

- Parameter allows you to specify variable width. But how do you write variable width hardware?
- Use `generate` and `genvar` to **generate units of hardware** in a **for loop**
- `genvar` is a loop variable that only exists at **elaboration time** (when the hardware is being built, before simulation runs)
- Eg: instantiate **N full_adder blocks** and wire them up

```
genvar i;
generate
  for (i = 0; i < N; i = i + 1) begin : bit_stage
    full_adder fa (
      .a(a[i]),
      .b(b[i]),
      .cin(carry[i]),
      .sum(sum[i]),
      .cout(carry[i+1])
    );
  end
endgenerate
```

Don’t keep looping over hardware!

- **Rule of thumb**: if you're wiring up **copies of a module** or **repeating assign statements**, use **`generate`**. If you're describing **what a circuit does** inside an always block, use a **regular for**.
- The latter is a behavioural description that runs at **simulation time**; the former generates hardware

```
// This creates 4 hardware AND gates
genvar i;
generate
  for (i = 0; i < 4; i = i + 1)
    assign out[i] = a[i] & b[i];
endgenerate

// This describes behaviour — runs every time inputs change
always @(*) begin
  for (i = 0; i < 4; i = i + 1)
    out[i] = a[i] & b[i];
end
```

Parameter is good, I like

- Parametric expressions can be used for indexing

```
wire feedback = crc_in[WIDTH-1];          // MSB, scales with WIDTH
wire [WIDTH-1:0] shifted = {crc_in[WIDTH-2:0], 1'b0};  // left shift by 1
```

- They can be used for filling out bits/replicating them. For eg: to initialize a register of size `WIDTH` to all ones: `parameter [WIDTH-1:0] INIT = {WIDTH{1'b1}};`
  - `{N{expr}}` means to repeat `expr`, N number of times and concatenate

- Remember **rule of thumb**: `generate/genvar` only help you when **instantiating parameterized hardware**, not when you're simply looping over indices, or using parametric indices etc.
- Above examples are proof that parametric DOES NOT IMPLY usage of generate.

It’s inlab (1) time

It’s graded lab time

1. Log in to:
   Username: **CS231-labexam**
   Password: …

2. Extract labexam tarball and work in the extracted folder

3. Go into the same folder and run
   `./build-archive.sh`

4. Log in to moodle and submit into the **correct activity: “Inlab Week 3 Assessment”**